第六章 贪心算法

Chapter 6 Greedy Algorithm

计算机学院 高翔
gaoxg@nwpu.edu.cn

目 录

4.1分支限界法基本思想

4.2 电子老鼠闯迷宫

4.3 跳马问题

4.4 独轮车问题

4.5 分油问题

4.6 八数码问题

4.7 单源最短路径问题

6.1 贪心算法的基本思想

找硬币

(0.25, 0.1, 0.05, 0.01)找0.63

(0.11, 0.05, 0.01）找0.15

贪心算法总是做出在当前看来最好的选择。

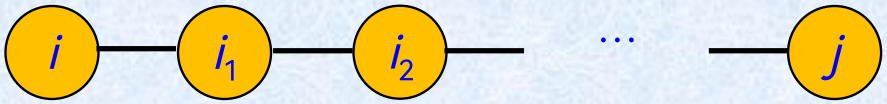
6.1.1 什么是贪心法

贪心法的基本思路是在对问题求解时总是做出在当前看来是最好的选择，也就是说贪心法不从整体最优上加以考虑，所做出的仅是在某种意义上的局部最优解。

人们通常希望找到整体最优解，所以采用贪心法需要证明设计的算法确实是整体最优解或求解了它要解决的问题。

例如，求解一个带权无向图G中从顶点i到顶点j（i≠j）的最短路径，可以分析出这样的最短路径一定是简单路径，所以约束条件为：

求解的路径为{(i，i1)，(i1，i2)，…，(im，j)|(i，i1)、(i1，i2)、…、(im，j)均为图G的边，且ik（1≤k≤m）均不相同}



目标函数就是要使这样的路径最短，即：

MIN pathlength {(i，i1)，(i1，i2)，…，(im，j)|

pathlength=w(i，i1)+w(i1，i2)+…+w(im，j)，

w(i，k)表示(i，k)的权值}

贪心法从问题的某一个初始解{}出发，采用逐步构造最优解的方法向给定的目标前进，每一步决策产生n-元组解（x_0，x_1，…，x_{n-1}）的一个分量。

贪心法每一步上用作决策依据的选择准则被称为最优量度标准（或贪心准则），也就是说，在选择解分量的过程中，添加新的解分量x_k后，形成的部分解（x_0，x_1，…，x_k)不违反可行解约束条件。

每一次贪心选择都将所求问题简化为规模更小的子问题，并期望通过每次所做的局部最优选择产生出一个全局最优解。

6.1.2 贪心法求解的问题应具有的性质

1. 贪心选择性质

所谓贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。

也就是说，贪心法仅在当前状态下做出最好选择，即局部最优选择，然后再去求解做出这个选择后产生的相应子问题的解。

2. 最优子结构性质

如果一个问题的最优解包含其子问题的最优解，则称此问题具有最优子结构性质。

问题的最优子结构性质是该问题可用动态规划算法或贪心法求解的关键特征。

6.1.3 贪心法的一般求解过程

贪心法求解问题的算法框架如下：

```
SolutionType Greedy(SType a[],int n)
//假设解向量(x0,x1,...,xn-1)类型为SolutionType，其分量为SType类型
{  SolutionType x={};                    //初始时，解向量不包含任何分量
   for (int i=0;i<n;i++)                 //执行n步操作
   {  SType xi=Select(a);                //从输入a中选择一个当前最好的分量
      if (Feasiable(xi))                 //判断xi是否包含在当前解中
            solution=Union(x,xi);        //将xi分量合并形成x
   }
   return x;                             //返回生成的最优解
}
```

6.2 活动安排问题

【问题描述】假设有一个需要使用某一资源的n个活动所组成的集合S，S={1，…，n}。该资源任何时刻只能被一个活动所占用，活动i有一个开始时间bi和结束时间ei（bi<ei），其执行时间为ei-bi，假设最早活动执行时间为0。

一旦某个活动开始执行，中间不能被打断，直到其执行完毕。若活动i和活动j有bi≥ej或bj≥ei，则称这两个活动兼容。

设计算法求一种最优活动安排方案，使得所有安排的活动个数最多。

【问题求解】假设活动时间的参考原点为0。一个活动i（$1 \le i \le n$）用一个区间$[b_i, e_i)$表示，当活动按结束时间（右端点）递增排序后，两个活动$[b_i, e_i)$和$[b_j, e_j)$兼容（满足$b_i \ge e_j$或$b_j \ge e_i$）实际上就是指它们不相交。

用数组A存放所有的活动，$A[i].b$（$1 \le i \le n$），存放活动起始时间，$A[i].e$存放活动结束时间。

例如，对于下表的n=11个活动（已按结束时间递增排序）A:

i	1	2	3	4	5	6	7	8	9	10	11
开始时间	1	3	0	5	3	5	6	8	8	2	12
结束时间	4	5	6	7	8	9	10	11	12	13	15

产生最大兼容活动集合的过程:

活动1 √
活动2 ×
活动3 ×
活动4 √
活动5 ×
活动6 ×
活动7 ×
活动8 √
活动9 ×
活动10 ×
活动11 √

最大兼容活动集合:

活动1	活动4	活动8	活动11

求解结果

```
//问题表示
struct Action                                   //活动的类型声明
{  int b;   //活动起始时间
   int e;   //活动结束时间
   bool operator<(const Action &s) const      //重载<关系函数
   {
            return e<=s.e;                     //用于按活动结束时间递增排序
   }
};
int n=11;
Action A[]={{0},{1,4},{3,5},{0,6},{5,7},{3,8},{5,9},{6,10},{8,11},
            {8,12},{2,13},{12,15}};        //下标0不用

//求解结果表示
bool flag[MAX];                 //标记选择的活动
int Count=0;                                    //选取的兼容活动个数
```

```
void solve()                                    //求解最大兼容活动子集
{  memset(flag,0,sizeof(flag));          //初始化为false
   sort(A+1,A+n+1);                             //A[1..n]按活动结束时间递增排序
   int preend=0;                             //前一个兼容活动的结束时间
   for (int i=1;i<=n;i++)                  //扫描所有活动
   {  if (A[i].b>=preend)                  //找到一个兼容活动
      {  flag[i]=true;              //选择A[i]活动
         preend=A[i].e;                      //更新preend值
      }
   }
}
```

【算法分析】算法的主要时间花费在排序上，排序时间为$O(n\log_2 n)$，所以整个算法的时间复杂度为$O(n\log_2 n)$。

【算法证明】通常证明一个贪心选择得出的解是最优解的一般的方法是，构造一个初始最优解，然后对该解进行修正，使其第一步为一个贪心选择，证明总是存在一个以贪心选择开始的求解方案。

对于本问题，所有活动按结束时间递增排序，就是要证明：若X是活动安排问题A的最优解，$X=X'\cup\{1\}$，则X'是$A'=\{i\in A:\ e_i\geq b_1\}$的活动安排问题的最优解。

首先证明总存在一个以活动1开始的最优解。

如果第一个选中的活动为k（$k\neq1$），可以构造另一个最优解Y，Y中的活动是兼容的，Y与X的活动数相同。

那么用活动1取代活动k得到Y'，因为$e_1\leq e_k$，所以Y'中的活动是兼容的，即Y'也是最优的，这就说明总存在一个以活动1开始的最优解。

当做出了对活动1的贪心选择后，原问题就变成了在活动2、…、n中找与活动1兼容的那些活动的子问题。亦即，如果X为原问题的一个最优解，则$X'=X-\{1\}$也是活动选择问题$A'=\{i\in A \mid b_i\geq e_1\}$的一个最优解。

反证法：如果能找到一个A'的含有比X'更多活动的解Y，则将活动1加入Y后就得到A的一个包含比X更多活动的解Y，这就与X是最优解的假设相矛盾。

因此，在每一次贪心选择后，留下的是一个与原问题具有相同形式的最优化问题，即最优子结构性质。

6.3 最优装载

有一批集装箱要装上一艘载重量为c的轮船。其中集装箱i的重量为Wi。最优装载问题要求确定在装载体积不受限制的情况下，将尽可能多的集装箱装上轮船。

1.算法描述

最优装载问题可用贪心算法求解。采用重量最轻者先装的贪心选择策略，可产生最优装载问题的最优解。具体算法描述如下页。

6.3.1 求解最优装载问题

【问题描述】有n个集装箱要装上一艘载重量为W的轮船，其中集装箱i（$1 \le i \le n$）的重量为w_i。

不考虑集装箱的体积限制，现要选出尽可能多的集装箱装上轮船，使它们的重量之和不超过W。

【问题求解】这里的最优解是选出尽可能多的集装箱个数，采用贪心法求解。

当重量限制为W时，w_i越小可装载的集装箱个数越多，所以采用优先选取重量轻的集装箱装船的贪心思路。

对w_i从小到大排序得到$\{w_1, w_2, \ldots, w_n\}$，设最优解向量为$x=\{x_1, x_2, \ldots, x_n\}$，显然，$x_1=1$，则$x'=\{x_2, \ldots, x_n\}$是装载问题$w'=\{w_2, \ldots, w_n\}$，$W'=W-w_1$的最优解，满足贪心最优子结构性质。

```
//问题表示
int w[]={0,5,2,6,4,3};                  //各集装箱重量,不用下标0的元素
int n=5,W=10;
//求解结果表示
int maxw;                               //存放最优解的总重量
int x[MAXN];                            //存放最优解向量

void solve()                            //求解最优装载问题
{  memset(x,0,sizeof(x));               //初始化解向量
   sort(w+1,w+n+1);                     //w[1..n]递增排序
   maxw=0;
   int restw=W;                         //剩余重量
   for (int i=1;i<=n && w[i]<=restw;i++)
   {  x[i]=1;                           //选择集装箱i
      restw-=w[i];                      //减少剩余重量
      maxw+=w[i];                       //累计装载总重量
   }
}
```

```
int w[]={0,5,2,6,4,3};      //各集装箱重量,不用下标0的元素
int n=5,W=10;
```

最优方案
  选取重量为2的集装箱
  选取重量为3的集装箱
  选取重量为4的集装箱
总重量=9

【算法分析】算法的主要时间花费在排序上，时间复杂度为$O(n\log_2 n)$。

6.3 背包问题

0-1背包问题:

给定n种物品和一个背包。物品i的重量是Wi，其价值为Vi，背包的容量为C。应如何选择装入背包的物品，使得装入背包中物品的总价值最大?

在选择装入背包的物品时，对每种物品i只有2种选择，即装入背包或不装入背包。不能将物品i装入背包多次，也不能只装入部分的物品i。

部分背包问题：

与0-1背包问题类似，所不同的是在选择物品i装入背包时，**可以选择物品i的一部分**，而不一定要全部装入背包，$1 \le i \le n$。

这2类问题都具有最优子结构性质，极为相似，但背包问题可以用贪心算法求解，而0-1背包问题却不能用贪心算法求解。

用贪心算法解背包问题的基本步骤：

首先计算每种物品单位重量的价值Vi/Wi，然后，依贪心选择策略，将尽可能多的单位重量价值最高的物品装入背包。若将这种物品全部装入背包后，背包内的物品总重量未超过C，则选择单位重量价值次高的物品并尽可能多地装入背包。依此策略一直地进行下去，直到背包装满为止。

部分 0-1

【问题描述】设有编号为1、2、…、n的n个物品，它们的重量分别为w_1、w_2、…、w_n，价值分别为v_1、v_2、…、v_n，其中w_i、v_i（$1 \le i \le n$）均为正数。

有一个背包可以携带的最大重量不超过W，求解**目标**：在不超过背包负重的前提下，使背包装入的总价值最大（即效益最大化），与0/1背包问题的区别是，这里的**每个物品可以取一部分装入背包**。

【问题求解】这里采用贪心法求解。设x_i表示物品i装入背包的情况，$0 \le x_i \le 1$。根据问题的要求，有如下约束条件和目标函数：

$$\sum_{i=1}^{n} w_i x_i \le W \quad 0 \le x_i \le 1 \ (1 \le i \le n)$$

$$\text{MAX}\{\sum_{i=1}^{n} v_i x_i\}$$

于是问题归结为寻找一个满足上述约束条件，并使目标函数达到最大的解向量$X=\{x_1, x_2, \ldots, x_n\}$。

例如，n=3，(w_1，w_2，w_3)=(18，15，10)，(v_1，v_2，v_3)=(25，24，15)，W=20，其中的4个可行解如下：

解编号	(x_1, x_2, x_3)	$\sum_{i=1}^{n} w_i x_i$	$\sum_{i=1}^{n} v_i x_i$
①	(1/2，1/3，1/4)	16.5	24.25
②	(1，2/15，0)	20	28.2
③	(0，2/3，1)	20	31
④	(0，1，1/2)	20	31.5

在这4个可行解中，第④个解的效益最大，可以求出它是这个背包问题的最优解。

贪心策略：选择单位重量价值最大的物品。

每次从物品集合中选择单位重量价值最大的物品，如果其重量小于背包容量，就可以把它装入，并将背包容量减去该物品的重量，然后就面临了一个最优子问题——它同样是背包问题，只不过背包容量减少了，物品集合减少了。

因此背包问题具有最优子结构性质。

对于下表一个背包问题，n=5，设背包容量W=100，其求解过程如下：

i	1	2	3	4	5
w_i	10	20	30	40	50
v_i	20	30	66	40	60
v_i/w_i	2.0	1.5	2.2	1.0	1.2

（1）将单位价值即v/w递减排序，其结果为{66/30，20/10，30/20，60/50，40/40}，物品重新按1～5编号。

i	1	2	3	4	5
w_i	30	10	20	50	40
v_i	66	20	30	60	40
v_i/w_i	2.2	2.0	1.5	1.2	1.0

（2）设背包余下装入的重量为weight=W。

i	1	2	3	4	5
w_i	30	10	20	50	40
v_i	66	20	30	60	40
v_i/w_i	2.2	2.0	1.5	1.2	1.0

（3）从i=1开始，w[1]<weight成立，表明物品1能够装入，将其装入到背包中，置x[1]=1，weight=weight-w[1]=70，i增1即i=2。

w[2]<weight成立，表明物品2能够装入，将其装入到背包中，置*x*[2]=1，weight=weight-*w*[2]=60，*i*增1即*i*=3。

w[3]<weight成立，表明物品3能够装入，将其装入到背包中，置*x*[3]=1，weight=weight-*w*[3]=50，*i*增1即*i*=4。

w[4]<weight不成立，且weight>0，表明只能将物品4部分装入，装入比例=weight/*w*[4]=40/50=80%，置*x*[4]=0.8。

算法结束，得到X={1，1，1，0.8，0}。

```
//问题表示
int n=5;
double W=100;                                   //限重
struct NodeType
{  double w;
   double v;
   double p;                                    //p=v/w
   bool operator<(const NodeType &s) const
   {
            return p>s.p;                       //按p递减排序
   }
};
NodeType A[]={{0},{10,20},{20,30},{30,66},{40,40},{50,60}};
                                                //下标0不用

//求解结果表示
double V;                                       //最大价值
double x[MAXN];
```

```
void Knap()                          //求解背包问题并返回总价值
{  V=0;                              //V初始化为0
   double weight=W;                  //背包中能装入的余下重量
   memset(x,0,sizeof(x));    //初始化x向量
   int i=1;

   while (A[i].w<weight)     //物品i能够全部装入时循环
   {  x[i]=1;                        //装入物品i
      weight-=A[i].w;                //减少背包中能装入的余下重量
      V+=A[i].v;             //累计总价值
      i++;                   //继续循环
   }

   if (weight>0)             //当余下重量大于0
   {  x[i]=weight/A[i].w;    //将物品i的一部分装入
      V+=x[i]*A[i].v;        //累计总价值
   }
}
```

```
void main()
{  printf("求解过程\n");
   for (int i=1;i<=n;i++)                    //求v/w
     A[i].p=A[i].v/A[i].w;
   printf("(1)排序前\n");dispA();
   sort(A+1,A+n+1);                                  //A[1..n]排序
   printf("(2)排序后\n"); dispA();
   Knap();
   printf("(3)求解结果\n");                  //输出结果
   printf("    x: [");
   for (int j=1;j<=n;j++)
      printf("%g, ",x[j]);
   printf("%g]\n",x[n]);
   printf("    总价值=%g\n",V);
}
```

【算法分析】排序的时间复杂性为$O(n\log_2 n)$，while循环的时间为$O(n)$，所以本算法的时间复杂度为$O(n\log_2 n)$。

6.4 求解多机调度问题

【问题描述】设有n个独立的作业{1，2，…，n}，由m台相同的机器{1，2，…，m}进行加工处理，作业i 所需的处理时间为t_i（$1 \le i \le n$），每个作业均可在任何一台机器上加工处理，但未完工前不允许中断，任何作业也不能拆分成更小的子作业。

多机调度问题要求给出一种作业调度方案，使所给的n个作业在尽可能短的时间内由m台机器加工处理完成。

【问题求解】贪心法求解多机调度问题的贪心策略是最长处理时间作业优先，即把处理时间最长的作业分配给最先空闲的机器，这样可以保证处理时间长的作业优先处理，从而在整体上获得尽可能短的处理时间。

按照最长处理时间作业优先的贪心策略，当$m \geq n$时，只要将机器i 的[0，t_j)时间区间分配给作业j即可；

当$m<n$时，首先将n个作业依其所需的处理时间从大到小排序，然后依此顺序将作业分配给空闲的处理机。

例如，有7个独立的作业{1，2，3，4，5，6，7}，由3台机器{1，2，3}加工处理，各作业所需的处理时间如下：

作业编号	1	2	3	4	5	6	7
作业的处理时间	2	14	4	16	6	5	3

采用贪心法求解的过程如下：

（1）7个作业按处理时间递减排序，其结果如下表所示。

作业编号	4	2	5	6	3	7	1
作业的处理时间	16	14	6	5	4	3	2

（2）先将排序后的前3个作业分配给3台机器。此时机器的分配情况为{{4}，{2}，{5}}，对应的总处理时间为{16，14，6}。

作业编号	4	2	5	6	3	7	1
作业的处理时间	16	14	6	5	4	3	2

机器1

作业4:16

机器2

作业2:14

机器3

作业5:6

（3）分配余下的作业：

作业编号	4	2	5	6	3	7	1
作业的处理时间	16	14	6	5	4	3	2

机器1

作业4，总时间16

机器2

作业2、7，总时间17

机器3

作业5、6、3、1，总时间17

作业调度方案

```
//问题表示
int n=7;
int m=3;
struct NodeType                                    //优先队列结点类型
{  int no; //作业序号
   int t;    //执行时间
   int mno;                                        //机器序号
   bool operator<(const NodeType &s) const
   {  return t>s.t;  }                             //按t越小越优先出队
};
struct NodeType A[]=
                {{1,2},{2,14},{3,4},{4,16},{5,6},{6,5},{7,3}};
```

```
void solve()                                        //求解多机调度问题
{  NodeType e;
   if (n<=m)
   {  printf("为每一个作业分配一台机器\n");
      return;
   }
   sort(A,A+n);                          //按t递减排序
   priority_queue<NodeType> qu; //小根堆
   for (int i=0;i<m;i++)                 //先分配m个作业，每台机器一个作业
   {  A[i].mno=i+1;                      //作业对应的机器编号
      printf("  给机器%d分配作业%d,执行时间为%2d,占用时间                    段:
[%d,%d]\n",A[i].mno,A[i].no,A[i].t,0,A[i].t);
      qu.push(A[i]);
   }
```

```
 for (int j=m;j<n;j++)                          //分配余下作业
   {  e=qu.top(); qu.pop();                     //出队e
      printf("  给机器%d分配作业%d,执行时间为%2d,占用时间                    段:
[%d,%d]\n",e.mno,A[j].no,A[j].t,e.t,e.t+A[j].t);
      e.t+=A[j].t;
      qu.push(e);                               //e进队
   }
}
```

【算法分析】排序的时间复杂度为O($n\log_2 n$)，两次for循环的时间合起来为O(n)，所以本算法的时间复杂度为O($n\log_2 n$)。

6.5 哈夫曼编码

【问题描述】设要编码的字符集为{d_1， d_2， …， d_n}，它们出现的频率为{w_1， w_2， …， w_n}，应用哈夫曼树构造最优的不等长的由0、1构成的编码方案。

【问题求解】先构建以这个n个结点为叶子结点的哈夫曼树，然后由哈夫曼树产生各叶子结点对应字符的哈夫曼编码。

哈夫曼树（Huffman Tree）的定义：设二叉树具有n个带权值的叶子结点，从根结点到每个叶子结点都有一个路径长度。从根结点到各个叶子结点的路径长度与相应结点权值的乘积的和称为该二叉树的带权路径长度，记作：

$$WPL = \sum_{i=1}^{n} w_i \times l_i$$

由n个叶子结点可以构造出多种二叉树，其中具有最小带权路径长度的二叉树称为哈夫曼树（也称最优树）。

构造一棵哈夫曼树的方法如下：

（1）由给定的n个权值$\{w_1, w_2, \ldots, w_n\}$构造$n$棵只有一个叶子结点的二叉树，从而得到一个二叉树的集合$F=\{T_1, T_2, \ldots, T_n\}$。

（2）在F中选取根结点的权值最小和次小的两棵二叉树作为左、右子树构造一棵新的二叉树，这棵新的二叉树根结点的权值为其左、右子树根结点权值之和。即合并两棵二叉树为一棵二叉树。

（3）重复步骤（2），当F中只剩下一棵二叉树时，这棵二叉树便是所要建立的哈夫曼树。

例如，给定的*a*~*e*的5个字符，它们的权值集合为*W*={4，2，1，7，3}，构造哈夫曼树的过程如下。

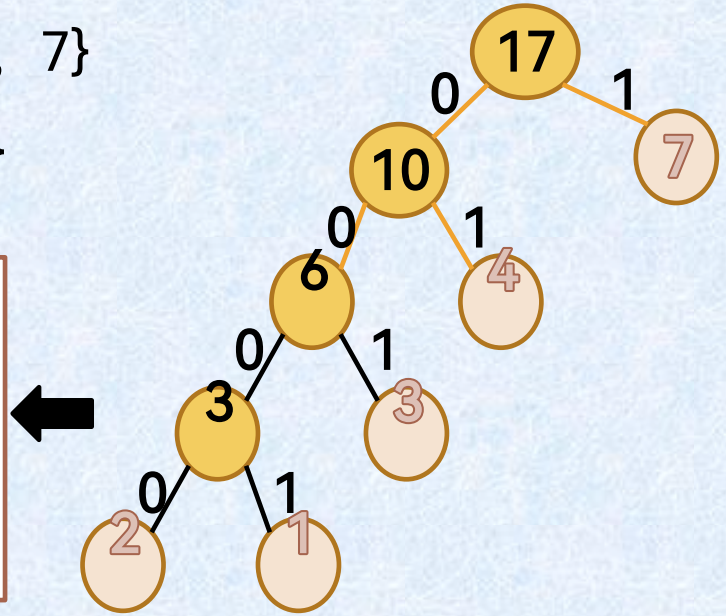
(1) {4，2，1，7，3} → {4，3，7，3}

(2) {4，3，7，3} → {4，6，7}

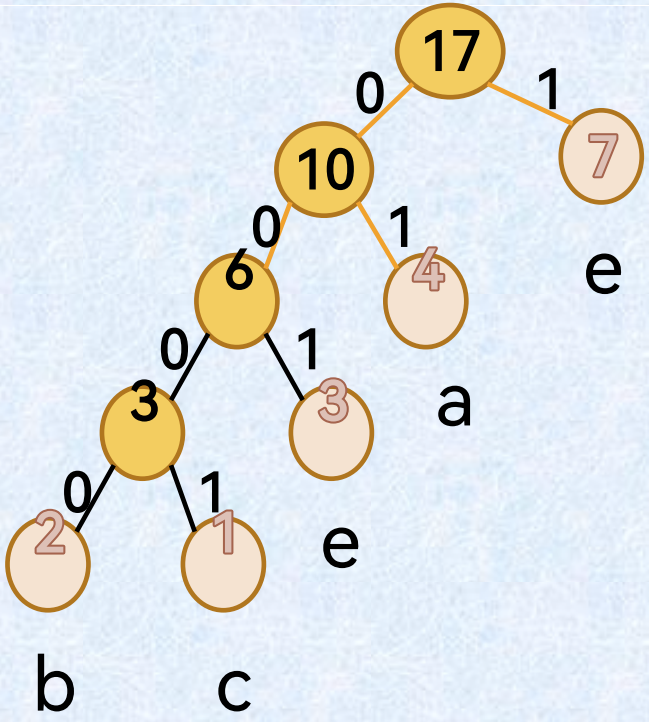
(3) {4，6，7} → {10，7}

(4) {10，7} → {17}

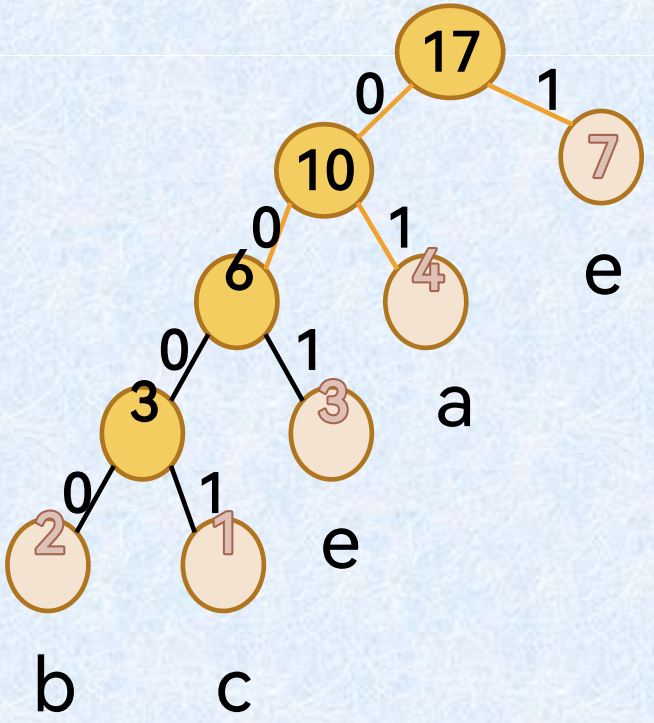
b(2)：0000　c(1)：0001
e(3)：001　a(4)：01
d(7)：1
WPL=(2+1)*4+3*3+4*2+7*1
=36



{4, 2, 1, 7, 3}



No	data	weight	parent	lchild	rchild
0	a	4	-1 7	-1	-1
1	b	2	-1 5	-1	-1
2	c	1	-1 5	-1	-1
3	d	7	-1 8	-1	-1
4	e	3	-1 6	-1	-1
5		3	-1 6	2	1
6		6	-1 7	5	4
7		10	-1 8	6	0
8		17	-1	7	3



```
int n;
struct HTreeNode                                    //哈夫曼树结点类型
{  char data;                                       //字符
   int weight;                                      //权值
   int parent;                                      //双亲的位置
   int lchild;                                      //左孩子的位置
   int rchild;                                      //右孩子的位置
};
HTreeNode ht[MAX];                                  //存放哈夫曼树
map<char,string> htcode;                 //存放哈夫曼编码

struct NodeType                                     //优先队列结点类型
{  int no; //对应哈夫曼树ht中的位置
   char data;                                       //字符
   int  weight;                                     //权值
   bool operator<(const NodeType &s) const
   {                                                //用于创建小根堆
            return s.weight<weight;
   }
};
```

```
void CreateHTree()                                  //构造哈夫曼树
{  NodeType e,e1,e2;
   priority_queue<NodeType> qu;
   for (int k=0;k<2*n-1;k++)                //设置所有结点的指针域
      ht[k].lchild=ht[k].rchild=ht[k].parent=-1;
   for (int i=0;i<n;i++)                    //将n个结点进队qu
   {  e.no=i; e.data=ht[i].data;
      e.weight=ht[i].weight; qu.push(e);
   }
```

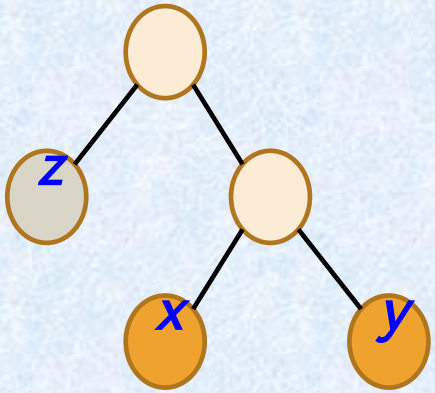
```
  for (int j=n;j<2*n-1;j++)                //构造哈夫曼树的n-1个非叶子结点
  {  e1=qu.top();  qu.pop();                        //出队权值最小的结点e1
     e2=qu.top();  qu.pop();               //出队权值次小的结点e2
     ht[j].weight=e1.weight+e2.weight; //构造哈夫曼树的非叶子结点j
     ht[j].lchild=e1.no;
     ht[j].rchild=e2.no;
     ht[e1.no].parent=j;                   //修改e1.no的双亲为结点j
     ht[e2.no].parent=j;                   //修改e2.no的双亲为结点j
     e.no=j;                                           //构造队列结点e
     e.weight=e1.weight+e2.weight;
     qu.push(e);
  }
}
```

```
void CreateHCode()                          //构造哈夫曼编码
{  string code;
   code.reserve(MAX);

   for (int i=0;i<n;i++)                    //构造叶子结点i的哈夫曼编码
   {  code="";
      int curno=i;
      int f=ht[curno].parent;
      while (f!=-1)                         //循环到根结点
      {  if (ht[f].lchild==curno)  //curno为双亲f的左孩子
            code='0'+code;
         else                                          //curno为双亲f的右孩子
            code='1'+code;
         curno=f; f=ht[curno].parent;
      }
      htcode[ht[i].data]=code;     //得到ht[i].data字符的哈夫曼
                                   编码
   }
}
```

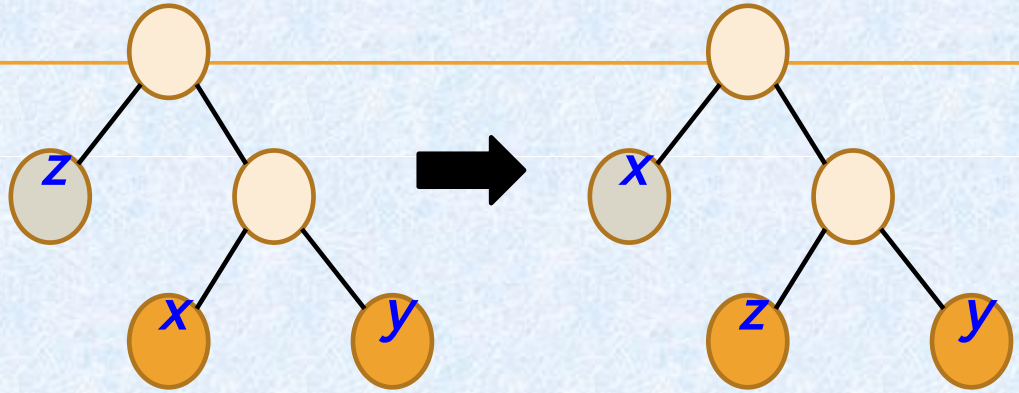
【算法证明】先讨论两个命题及其证明过程。

命题1：两个最小权值字符对应的结点x和y必须是哈夫曼树中最深的两个结点且它们为兄弟。



证明：假设x结点在哈夫曼树（最优树）中不是最深的，那么存在一个结点z，有$w_z > w_x$，但它比x深，即$l_z > l_x$。

此时x和z的带权和为$w_x \times l_x + w_z \times l_z$。



如果交换x和z结点的位置，其他不变

⇒ 交换后的带权和为$w_x \times l_z + w_z \times l_x$。

有：$w_x \times l_z + w_z \times l_x < w_x \times l_x + w_z \times l_z$

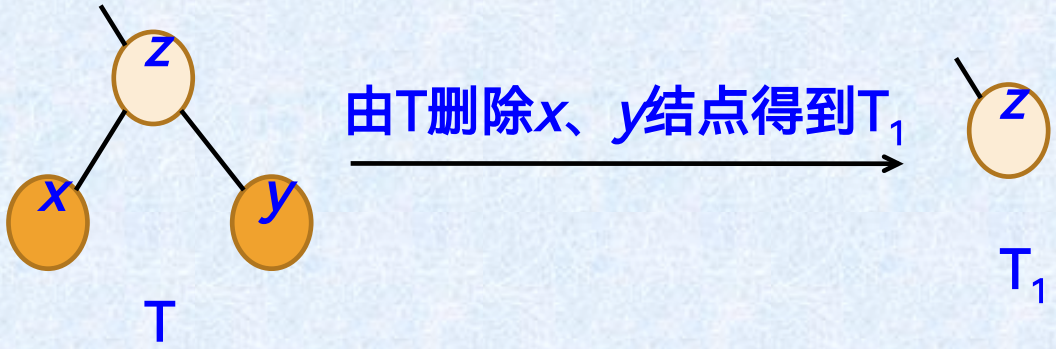
这是因为 $w_x \times l_z + w_z \times l_x - (w_x \times l_x + w_z \times l_z)$

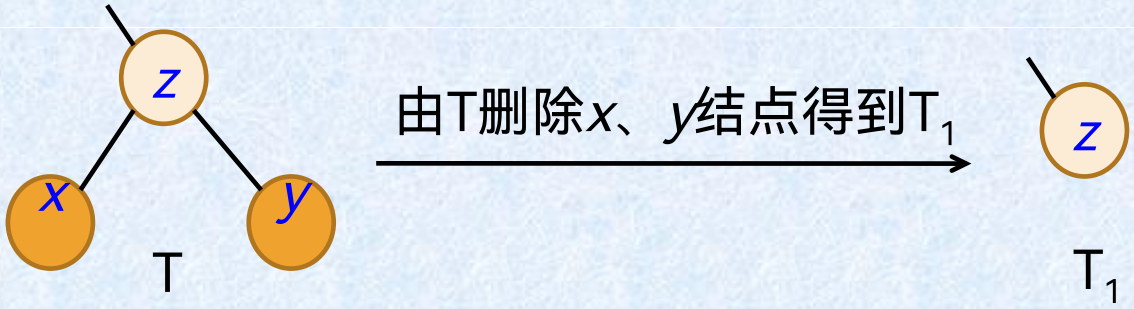
$= w_x(l_z - l_x) - w_z(l_z - l_x)$

$= (w_x - w_z)(l_z - l_x) < 0$（由前面所设有$w_z > w_x$和$l_z > l_x$）。

这就与交换前的树是最优树的假设矛盾。所以上述命题成立。

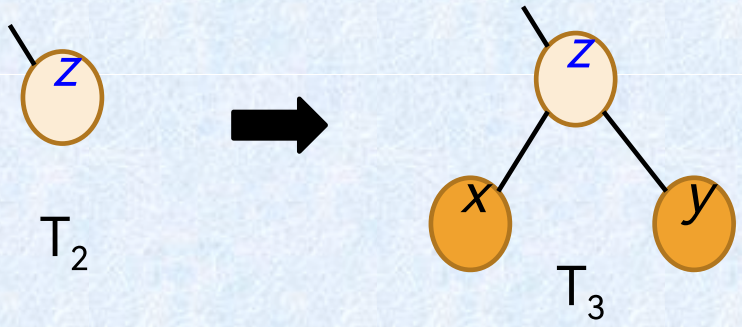
命题2：设T是字符集C对应的一棵哈夫曼树，结点x和y是兄弟，它们的双亲为z，显然有$w_z = w_x + w_y$，现删除结点x和y，让z变为叶子结点，那么这棵新树T_1一定是字符集$C_1 = C - \{x, y\} \cup \{z\}$的最优树。





证明：设T和T_1的带权路径长度分别为WPL(T)和WPL(T_1)，则有：$WPL(T) = WPL(T_1)+w_x+w_y$

这是因为WPL(T_1)含有T中除x、y外的所有叶子结点的带权路径长度和，另加上z的带权路径长度。



假设T_1不是最优的，则存在另一棵树T_2，有：

$$WPL(T_2) < WPL(T_1)$$

由于$z \in C_1$，则z在T_2中一定是一个叶子结点。若将x和y加入T_2中作为结点z的左、右孩子，则得到表示字符集C的前缀树T_3

且有：$WPL(T_3) = WPL(T_2)+w_x+w_y$

由前面几个式子看到

$$WPL(T_3) = WPL(T_2)+w_x + w_y<WPL(T_1)+w_x+w_y = WPL(T)$$

这与T为C的哈夫曼树的假设矛盾。本命题即证。

命题1说明该算法满足贪心选择性质，即通过合并来构造一棵哈夫曼树的过程可以从合并两个权值最小的字符开始。

命题2说明该算法满足最优子结构性质，即该问题的最优解包含其子问题的最优解。所以采用哈夫曼树算法产生的树一定是一棵最优树。

【算法分析】上述算法采用了小根堆，因为从堆中删除两个结点（权值最小的两个二叉树根结点）和加入一个新结点的时间复杂度是$O(\log_2 n)$，这样修改后构造哈夫曼树算法的时间复杂度为$O(n\log_2 n)$。

6.6 求解流水作业调度问题

【问题描述】有n个作业（编号为1～n）要在由两台机器M1和M2组成的流水线上完成加工。每个作业加工的顺序都是先在M1上加工，然后在M2上加工。M1和M2加工作业i所需的时间分别为a_i和b_i（$1 \le i \le n$）。

流水作业调度问题要求确定这n个作业的最优加工顺序，使得从第一个作业在机器M1上开始加工，到最后一个作业在机器M2上加工完成所需的时间最少。可以假定任何作业一旦开始加工，就不允许被中断，直到该作业被完成，即非优先调度。

【问题求解】采用一种称为Johnson算法的贪心算法。其步骤如下：

（1）把所有作业按M1、M2的时间分为两组，$a[i] \le b[i]$对应第1组N1，$a[i] > b[i]$对应第0组N2。

（2）将N1的作业按$a[i]$递增排序，N2的作业按$b[i]$递减排序。

（3）按顺序先执行N1的作业，再执行N2的作业，得到的就是耗时最少的最优调度方案。

实际上，N2的作业也按b[i]递增排序，从后面向前面顺序执行

示例：n=4

编号	1	2	3	4
M1	5	12	4	8
M2	6	2	14	7

（1）把所有作业按M1、M2的时间分为两组，$a[i] \le b[i]$对应第1组N1，$a[i] > b[i]$对应第0组N2。

组号	1	0	1	0
时间time	5	2	4	7

（2）将N1（组号=1， $a[i] \le b[i]$ ）的作业按$a[i]$（用时间time存放）递增排序，N2 （组号=1， $a[i] > b[i]$ ）的作业按$b[i]$（用时间time存放）递增排序。

编号	1	2	3	4
M1	5	12	4	8
M2	6	2	14	7
组号	1	0	1	0
时间time	5	2	4	7

编号	2	3	1	4
M1	12	4	5	8
M2	2	14	6	7
组号	0	1	1	0
时间time	2	4	5	7

（3）按顺序先执行N1的作业（顺序），再执行N2的作业（反序），得到的就是耗时最少的最优调度方案。

编号	2	3	1	4
M1	12	4	5	8
M2	2	14	6	7
组号	0	1	1	0
时间time	2	4	5	7

编号	3	1	4	2
M1	4	5	8	12
M2	14	6	7	2
组号	1	1	0	0
时间time	4	5	7	2

最优调度方案：3 1 4 2

求在最优调度下总时间，用f1累计M1上的执行时间（初始时f1=0）；f2累计M2上的执行时间（初始时f2=0），最终f2即为最优调度下的消耗总时间。

对于最优调度方案best，用*i*扫描best的元素，f1和f2的计算如下：

f1=f1+*a*[best[*i*]]

f2=max{f1，f2}+*b*[best[*i*]]

编号	3	1	4	2
M1	4	5	8	12
M2	14	6	7	2
组号	1	1	0	0
时间time	4	5	7	2

f1=f2=0

作业3：f1=0+4=4，f2=max{4,0}+14=18

作业1：f1=4+5=9，f2=max{9,18}+6=24

作业4：f1=9+8=17，f2=max{17,24}+7=31

作业2：f1=17+12=29，f2=max{29,31}+2=33

其实现采用如下结构体数组c:

```
struct NodeType
{  int no;                          //作业序号
   bool group;                      //1代表第一组N1,0代表第二
组N2
   int time;                        //a,b的最小时间
   bool operator<(const NodeType &s) const
   {
          return time<s.time;     //用于按time递增排序
   }
};
```

如

编号	1	2	3	4
M1	5	12	4	8
M2	6	2	14	7
组号	1	0	1	0
时间time	5	2	4	7

```
//问题表示
int n=4;
int a[N]={5,12,4,8};                    //对应M1的时间
int b[N]={6,2,14,7};                    //对应M2的时间
struct NodeType
{  int no; //作业序号
   bool group;                          //1代表第一组N1,0代表第二组N2
   int time;                            //a,b的最小时间
   bool operator<(const NodeType &s) const
   {
     return time<s.time;       //按time递增排序
   }
};

//求解结果表示
int best[N];                            //最优调度序列
```

```
int solve()                          //求解流水作业调度问题
{  int i,j,k;
   NodeType c[N];
   for(i=0;i<n;i++)                  //n个作业中,求出每个作业的最小加工时间
   {  c[i].no=i;
      c[i].group=(a[i]<=b[i]);
                        //a[i]<=b[i]对应第1组N1,a[i]>b[i]对应第0组N2
      c[i].time=a[i]<=b[i]?a[i]:b[i];
                                    //第1组存放a[i],第0组存放b[i]
   }

   sort(c,c+n);     //c元素按time递增排序
```

```
    j=0; k=n-1;
    for(i=0;i<n;i++)                    //扫描c所有元素,产生最优调度方案
    {  if(c[i].group==1)  //第1组,按time递增排列放在best的前面部分
          best[j++]=c[i].no;
       else                             //第0组,按time递减排列放到best的后面部分
          best[k--]=c[i].no;
    }
    int f1=0;                                   //累计M1上的执行时间
    int f2=0;                                   //最优调度下的消耗总时间
    for(i=0;i<n;i++)
    {  f1+=a[best[i]];
       f2=max(f2,f1)+b[best[i]];
    }
    return f2;
}
```

写得好·

```
void main()
{  printf("求解结果\n");
   printf("    总时间: %d\n",solve());
   printf("    调度方案: ");
   for(int i=0;i<n;i++)
     printf("%d ",best[i]+1);
   printf("\n");
}
```

编号	1	2	3	4
M1	5	12	4	8
M2	6	2	14	7

求解结果

总时间: 33

调度方案: 3 1 4 2

【算法分析】算法的主要时间花费在排序上，所以时间复杂度为$O(n\log_2 n)$。比采用回溯法和分枝限界法求解更高效。

6.7 畜栏问题

【例】求解畜栏问题。农场有n头牛，每头牛会有一个特定的时间区间[b，e]在畜栏里挤牛奶，并且一个畜栏里任何时刻只能有一头牛挤奶。

现在农场主希望知道最少畜栏能够满足上述要求，并给出每头牛被安排的方案。对于多种可行方案，输出一种即可。

i	1	2	3	4	5	6	7
开始时间	1	2	5	8	4	12	11
结束时间	4	5	7	9	10	13	15

1	3	4	6	2	7	5	
1	5	8	12	2	11	4	
4	7	9	13	5	15	10	

最大兼容活动子集个数为3

解：牛的编号为1～n，每头牛的挤奶时间相当于一个活动，与前面活动安排问题不同，这里的活动时间是闭区间，例如[2，4]与[4，7]是交叉的，它们不是兼容活动。

采用与求解活动安排问题类似的贪心思路，将所有活动这样排序：结束时间相同按开始时间递增排序，否则按结束时间递增排序。

求出一个最大兼容活动子集，将它们安排在一个畜栏中（畜栏编号为1）；如果没有安排完，再在剩余的活动再求下一个最大兼容活动子集，将它们安排在另一个畜栏中（畜栏编号为2），以此类推。

也就是说，最大兼容活动子集的个数就是最少畜栏个数。

```
//问题表示
struct Cow                                  //奶牛的类型声明
{  int no; //牛编号
   int b;   //起始时间
   int e;   //结束时间
   bool operator<(const Cow &s) const   //重载<关系函数
   {  if (e==s.e)                  //结束时间相同按开始时间递增排序
         return b<=s.b;
      else                         //否则按结束时间递增排序
         return e<=s.e;
   }
};
int n=5;
Cow A[]={{0},{1,1,10},{2,2,4},{3,3,6},{4,5,8},{5,4,7}};
                                            //下标0不用
//求解结果表示
int ans[MAX];                               //ans[i]表示第A[i].no头牛的畜栏编号
```

```
void solve()                                    //求解最大兼容活动子集个数
{  sort(A+1,A+n+1);                             //A[1..n]按指定方式排序
   memset(ans,0,sizeof(ans));                   //初始化为0
   int num=1;                                   //畜栏编号

   for (int i=1;i<=n;i++)                       //i、j均为排序后的下标
   {  if (ans[i]==0)                            //第i头牛还没有安排畜栏
      {  ans[i]=num;                            //第i头牛安排畜栏num
         int preend=A[i].e;                     //前一个兼容活动的结束时间
         for (int j=i+1;j<=n;j++)               //查找一个最大兼容活动子集
         {  if (A[j].b>preend && ans[j]==0)
            {  ans[j]=num;          //将兼容活动子集中活动安排在num畜栏中
               preend=A[j].e;                   //更新结束时间
            }
         }
         num++;                     //查找下一个最大兼容活动子集,num增1
      }
   }
}
```

```
void main()
{  solve();
   printf("求解结果\n");
   for (int i=1;i<=n;i++)
      printf("    牛%d安排的畜栏:%d\n",A[i].no,ans[i]);
}
```

i	1	2	3	4	5
开始时间	1	2	3	5	4
结束时间	10	4	6	8	7

求解结果

牛2安排的蓄栏:1
牛3安排的蓄栏:2
牛5安排的蓄栏:3
牛4安排的蓄栏:1
牛1安排的蓄栏:4